

Joining Tables

In this chapter, you'll learn what joins are, why they are used, and how to create `SELECT` statements using them.

Understanding Joins

One of SQL's most powerful features is the capability to join tables on-the-fly within data retrieval queries. Joins are some of the most important operations you can perform using SQL `SELECT`, and a good understanding of joins and join syntax is an extremely important part of learning SQL.

Before you can effectively use joins, you must understand relational tables and the basics of relational database design. What follows is by no means complete coverage of the subject, but it should be enough to get you up and running.

Understanding Relational Tables

The best way to understand relational tables is to look at a real-world example.

Suppose you have a database table containing a product catalog, with each catalog item in its own row. The information you store with each item includes a product description and price, along with vendor information about the company that creates the product.

Now suppose you have multiple catalog items created by the same vendor. Where would you store the vendor information—things such as vendor name, address, and contact information? You wouldn't want to store that data along with the products for several reasons:

- Because the vendor information is the same for each product the vendor produces, repeating the information for each product would be a waste of time and storage space.
- If vendor information changes (for example, if the vendor moves or contact info changes), you would need to update every occurrence of the vendor information.
- When data is repeated (that is, when the vendor information is used with each product), there is a high likelihood that the data will not be entered exactly the same way each time. Inconsistent data is extremely difficult to use in reporting.

The key here is that having multiple occurrences of the same data is never a good thing, and this principle is the basis for relational database design. Relational tables are designed so information is split into multiple tables, one for each data type. The tables are related to each other through common values (and thus the *relational* in relational design).

In our example, you can create two tables: one for vendor information and one for product information. The `vendors` table contains all the vendor information, one table row per vendor, along with a unique identifier for each vendor. This value, called a *primary key*, can be a vendor ID or any other unique value. (Primary keys are first mentioned in Chapter 1, “Understanding SQL.”)

The `products` table stores only product information and no vendor-specific information other than the vendor ID (the `vendors` table’s primary key). This key, called a *foreign key*, relates the `vendors` table to the `products` table, and using this vendor ID enables you to use the `vendors` table to find details about the appropriate vendor.

New Term

Foreign Key A column in one table that contains the primary key values from another table, thus defining the relationships between tables.

What does this do for you? Well, consider the following:

- Vendor information is never repeated, and so time and space are not wasted.
- If vendor information changes, you can update a single record in the `vendors` table. Data in related tables does not change.
- As no data is repeated, the data used is obviously consistent, making data reporting and manipulation much simpler.

The bottom line is that relational data can be stored efficiently and manipulated easily. Because of this, relational databases scale far better than non-relational databases.

New Term

Scale To be able to handle an increasing load without failing. A well-designed database or application is said to *scale well*.

Why Use Joins?

As just explained, breaking data into multiple tables enables more efficient storage, easier manipulation, and greater scalability. But these benefits come with a price.

If data is stored in multiple tables, how can you retrieve that data with a single `SELECT` statement?

The answer is to use a join. Simply put, a *join* is a mechanism used to associate (or join) tables within a `SELECT` statement (thus the name join). By using special syntax, you can join multiple tables so a single set of output is returned, and the join associates the correct rows in each table on-the-fly.

Note

Maintaining Referential Integrity It is important to understand that a join is not a physical entity; in other words, it does not exist in the actual database tables. MySQL creates joins as needed, and a join persists for the duration of the query execution.

When using relational tables, it is important that only valid data is inserted into relational columns. Going back to our example, if products were stored in the `products` table with an invalid vendor ID (one not present in the `vendors` table), those products would be inaccessible because they would not be related to any vendor.

To prevent this from occurring, you can instruct MySQL to allow only valid values (ones present in the `vendors` table) in the vendor ID column in the `products` table. This is known as maintaining *referential integrity*, and is achieved by specifying the primary and foreign keys as part of the table definitions (as will be explained in Chapter 21, “Creating and Manipulating Tables”).

Creating a Join

Creating a join is very simple. You must specify all the tables to be included and how they are related to each other. Look at the following example:

► Input

```
SELECT vend_name, prod_name, prod_price
FROM vendors, products
WHERE vendors.vend_id = products.vend_id
ORDER BY vend_name, prod_name;
```

► Output

vend_name	prod_name	prod_price
ACME	Bird seed	10.00
ACME	Carrots	2.50
ACME	Detonator	13.00
ACME	Safe	50.00
ACME	Sling	4.49
ACME	TNT (1 stick)	2.50
ACME	TNT (5 sticks)	10.00
Anvils R Us	.5 ton anvil	5.99
Anvils R Us	1 ton anvil	9.99
Anvils R Us	2 ton anvil	14.99
Jet Set	JetPack 1000	35.00
Jet Set	JetPack 2000	55.00
LT Supplies	Fuses	3.42
LT Supplies	Oil can	8.99

► Analysis

In this code, the `SELECT` statement starts the same way as all the statements you’ve looked at thus far: by specifying the columns to be retrieved. The big difference here is that two of the specified columns (`prod_name` and `prod_price`) are in one table, whereas the other (`vend_name`) is in another table.

Now look at the `FROM` clause. Unlike all the prior `SELECT` statements, this one has two tables listed in the `FROM` clause: `vendors` and `products`. These are the names of the two tables that are being joined in this `SELECT` statement. The tables are correctly joined with a `WHERE` clause that instructs MySQL to match `vend_id` in the `vendors` table with `vend_id` in the `products` table.

Notice that the columns are specified as `vendors.vend_id` and `products.vend_id`. Fully qualified column names are required here because if you just specify `vend_id`, MySQL will not be able to tell which `vend_id` column you are referring to (as there are two of them, one in each table).

Caution

Fully Qualifying Column Names You must use the fully qualified column name (table name and column name separated by a period) whenever there is possible ambiguity about the column you are referring to. MySQL returns an error message if you refer to an ambiguous column name without fully qualifying it with a table name.

The Importance of the WHERE Clause

It might seem strange to use a `WHERE` clause to set the join relationship, but actually, there is a very good reason for this. Remember that when tables are joined in a `SELECT` statement, the relationship is constructed on-the-fly. Nothing in the database table definitions can instruct MySQL how to join the tables. You have to do that yourself. When you join two tables, what you are actually doing is pairing every row in the first table with every row in the second table. The `WHERE` clause acts as a filter to include only rows that match the specified filter condition—the join condition, in this case. Without the `WHERE` clause, every row in the first table is paired with every row in the second table, regardless of whether they logically go together.

To understand this, look at the following `SELECT` statement and its output:

► Input

```
SELECT vend_name, prod_name, prod_price
FROM vendors, products
ORDER BY vend_name, prod_name;
```

► Output

vend_name	prod_name	prod_price
ACME	.5 ton anvil	5.99
ACME	1 ton anvil	9.99
ACME	2 ton anvil	14.99

ACME	Bird seed	10.00	
ACME	Carrots	2.50	
ACME	Detonator	13.00	
ACME	Fuses	3.42	
ACME	JetPack 1000	35.00	
ACME	JetPack 2000	55.00	
ACME	Oil can	8.99	
ACME	Safe	50.00	
ACME	Sling	4.49	
ACME	TNT (1 stick)	2.50	
ACME	TNT (5 sticks)	10.00	
Anvils R Us	.5 ton anvil	5.99	
Anvils R Us	1 ton anvil	9.99	
Anvils R Us	2 ton anvil	14.99	
Anvils R Us	Bird seed	10.00	
Anvils R Us	Carrots	2.50	
Anvils R Us	Detonator	13.00	
Anvils R Us	Fuses	3.42	
Anvils R Us	JetPack 1000	35.00	
Anvils R Us	JetPack 2000	55.00	
Anvils R Us	Oil can	8.99	
Anvils R Us	Safe	50.00	
Anvils R Us	Sling	4.49	
Anvils R Us	TNT (1 stick)	2.50	
Anvils R Us	TNT (5 sticks)	10.00	
Furball Inc.	.5 ton anvil	5.99	
Furball Inc.	1 ton anvil	9.99	
Furball Inc.	2 ton anvil	14.99	
Furball Inc.	Bird seed	10.00	
Furball Inc.	Carrots	2.50	
Furball Inc.	Detonator	13.00	
Furball Inc.	Fuses	3.42	
Furball Inc.	JetPack 1000	35.00	
Furball Inc.	JetPack 2000	55.00	
Furball Inc.	Oil can	8.99	
Furball Inc.	Safe	50.00	
Furball Inc.	Sling	4.49	
Furball Inc.	TNT (1 stick)	2.50	
Furball Inc.	TNT (5 sticks)	10.00	
Jet Set	.5 ton anvil	5.99	
Jet Set	1 ton anvil	9.99	
Jet Set	2 ton anvil	14.99	
Jet Set	Bird seed	10.00	
Jet Set	Carrots	2.50	
Jet Set	Detonator	13.00	
Jet Set	Fuses	3.42	
Jet Set	JetPack 1000	35.00	
Jet Set	JetPack 2000	55.00	
Jet Set	Oil can	8.99	

Jet Set	Safe	50.00	
Jet Set	Sling	4.49	
Jet Set	TNT (1 stick)	2.50	
Jet Set	TNT (5 sticks)	10.00	
Jouets Et Ours	.5 ton anvil	5.99	
Jouets Et Ours	1 ton anvil	9.99	
Jouets Et Ours	2 ton anvil	14.99	
Jouets Et Ours	Bird seed	10.00	
Jouets Et Ours	Carrots	2.50	
Jouets Et Ours	Detonator	13.00	
Jouets Et Ours	Fuses	3.42	
Jouets Et Ours	JetPack 1000	35.00	
Jouets Et Ours	JetPack 2000	55.00	
Jouets Et Ours	Oil can	8.99	
Jouets Et Ours	Safe	50.00	
Jouets Et Ours	Sling	4.49	
Jouets Et Ours	TNT (1 stick)	2.50	
Jouets Et Ours	TNT (5 sticks)	10.00	
LT Supplies	.5 ton anvil	5.99	
LT Supplies	1 ton anvil	9.99	
LT Supplies	2 ton anvil	14.99	
LT Supplies	Bird seed	10.00	
LT Supplies	Carrots	2.50	
LT Supplies	Detonator	13.00	
LT Supplies	Fuses	3.42	
LT Supplies	JetPack 1000	35.00	
LT Supplies	JetPack 2000	55.00	
LT Supplies	Oil can	8.99	
LT Supplies	Safe	50.00	
LT Supplies	Sling	4.49	
LT Supplies	TNT (1 stick)	2.50	
LT Supplies	TNT (5 sticks)	10.00	
+-----+			

► Analysis

This output shows every product matched with every vendor, including products with the incorrect vendor (and even vendors with no products at all). This is a *Cartesian product*, and it is seldom what you want.

New Term

Cartesian Product The results returned by a table relationship without a join condition. The number of rows retrieved is the number of rows in the first table multiplied by the number of rows in the second table.

Caution

Don't Forget the WHERE Clause Make sure all your joins have `WHERE` clauses, or MySQL returns far more data than you want. Similarly, make sure your `WHERE` clauses are correct. An incorrect filter condition causes MySQL to return incorrect data.

Note

Cross Joins Sometimes you'll hear the type of join that returns a Cartesian product referred to as a *cross join*.

Inner Joins

The type of join you have been using so far—a join based on the testing of equality between two tables—is called an *equijoin*. This kind of join is also called an *inner join*. In fact, you may use slightly different syntax for these joins, specifying the type of join explicitly. The following `SELECT` statement returns exactly the same data as the preceding example:

► Input

```
SELECT vend_name, prod_name, prod_price
FROM vendors INNER JOIN products
ON vendors.vend_id = products.vend_id;
```

► Analysis

The `SELECT` in the statement is the same as the preceding `SELECT` statement, but the `FROM` clause is different. Here the relationship between the two tables is part of the `FROM` clause specified as `INNER JOIN`. When using this syntax, the join condition is specified using the special `ON` clause instead of a `WHERE` clause. The actual condition passed to `ON` is the same as would be passed to `WHERE`.

Note

Which Syntax to Use? Per the ANSI SQL specification, the `INNER JOIN` syntax is preferable. Furthermore, although using the `WHERE` clause to define joins is simpler, using explicit join syntax ensures that you will never forget the join condition, and it can affect performance, too (in some cases). But, that said, the simpler `WHERE` clause is supported, so feel free to use it if you prefer.

Joining Multiple Tables

SQL imposes no limit to the number of tables that may be joined in a `SELECT` statement. The basic rules for creating a join remain the same: First list all the tables and then define the relationships between them. Here is an example:

► Input

```
SELECT prod_name, vend_name, prod_price, quantity
FROM orderitems, products, vendors
WHERE products.vend_id = vendors.vend_id
      AND orderitems.prod_id = products.prod_id
      AND order_num = 20005;
```

► Output

prod_name	vend_name	prod_price	quantity
.5 ton anvil	Anvils R Us	5.99	10
1 ton anvil	Anvils R Us	9.99	3
TNT (5 sticks)	ACME	10.00	5
Bird seed	ACME	10.00	1

► Analysis

This example displays the items in order number 20005. Order items are stored in the `orderitems` table. Each product is stored by its product ID, which refers to a product in the `products` table. The products are linked to the appropriate vendors in the `vendors` table by vendor ID, which is stored with each product record. The `FROM` clause here lists the three tables, and the `WHERE` clause defines both of those join conditions. An additional `WHERE` condition is then used to filter just the items for order 20005.

Here is the `INNER JOIN` version of the same `SELECT` statement:

► Input

```
SELECT prod_name, vend_name, prod_price, quantity
FROM vendors
INNER JOIN products
      ON vendors.vend_id = products.vend_id
INNER JOIN orderitems
      ON orderitems.prod_id = products.prod_id
WHERE order_num = 20005;
```

► Analysis

In this version, the `SELECT` is the same, as is the final `WHERE` clause. The difference is in how the tables are defined and joined together. Here, `vendors` is the `FROM` table, and the two additional tables are included and joined using `INNER JOIN` and an `ON` to define the relationship. The syntax is different, but the results will be identical.

Caution

Performance Considerations MySQL processes joins at runtime, relating each table as specified. This process can become very resource intensive, so be careful not to join tables unnecessarily. The more tables you join, the more performance degrades.

Now would be a good time to revisit an example from Chapter 14, “Working with Subqueries.” As you will recall, this `SELECT` statement returns a list of customers who ordered product TNT2:

► Input

```
SELECT cust_name, cust_contact
FROM customers
WHERE cust_id IN (SELECT cust_id
                  FROM orders
                  WHERE order_num IN (SELECT order_num
                                      FROM orderitems
                                      WHERE prod_id = 'TNT2'));
```

As mentioned in Chapter 14, using subqueries might not always be the most efficient way to perform complex `SELECT` operations, and so, as promised, here is the same query using joins:

► Input

```
SELECT cust_name, cust_contact
FROM customers, orders, orderitems
WHERE customers.cust_id = orders.cust_id
   AND orderitems.order_num = orders.order_num
   AND prod_id = 'TNT2';
```

► Output

cust_name	cust_contact
Coyote Inc.	Y Lee
Yosemite Place	Y Sam

► Analysis

As explained in Chapter 14, returning the data needed in this query requires the use of three tables. But instead of using them within nested subqueries, here two joins are used to connect the tables. There are three `WHERE` clause conditions here. The first two connect the tables in the join, and the last one filters the data for product TNT2.

For the sake of comparison, here is the `INNER JOIN` version of the same statement:

► Input

```
SELECT cust_name, cust_contact
FROM customers
INNER JOIN orders
  ON customers.cust_id = orders.cust_id
INNER JOIN orderitems
  ON orderitems.order_num = orders.order_num
WHERE prod_id = 'TNT2';
```

► Output

cust_name	cust_contact
Coyote Inc.	Y Lee
Yosemite Place	Y Sam

Tip

It Pays to Experiment As you can see, there is often more than one way to perform any given SQL operation. And there is rarely a definitive right or wrong way. Performance can be affected by the type of operation, the amount of data in the tables, whether indexes and keys are present, and a whole slew of other criteria. Therefore, it is often worth experimenting with different selection mechanisms to find the one that works best in a situation.

Summary

Joins are some of the most important and powerful operations in SQL. To use them effectively, you need a basic understanding of relational database design. In this chapter, you learned some of the basics of relational database design as an introduction to learning about joins. You also learned how to create an equijoin (also known as an inner join), which is the most commonly used form of join. In the next chapter, you'll learn how to create other types of joins.

Challenges

1. Write a SQL statement to return the customer name (`cust_name`) from the `Customers` table and related order numbers (`order_num`) from the `Orders` table and sort the result by customer name and then by order number. Actually, try this one twice: once using simple equijoin syntax and then using `INNER JOIN`.

2. Let's make the previous challenge more useful. In addition to returning the customer name and order number, add a third column named `OrderTotal` that contains the total price of each order. There are two ways to do this: You can create the `OrderTotal` column using a subquery on the `OrderItems` table, or you can join the `OrderItems` table to the existing tables and use an aggregate function. Here's a hint: Watch out for where you need to use fully qualified column names.
3. Let's revisit Challenge 2 from Chapter 14. Write a SQL statement that retrieves the dates when product BR01 was ordered, but this time use a join and simple equijoin syntax. The output should be identical to the output in the Chapter 14 challenge.
4. That was fun; let's try it again. Re-create the SQL you wrote for Challenge 3 from Chapter 14 but this time using ANSI `INNER JOIN` syntax. The code you wrote there employed two nested subqueries. To re-create it, you'll need two `INNER JOIN` statements, each formatted like the `INNER JOIN` example earlier in this chapter. And don't forget the `WHERE` clause to filter by `prod_id`.
5. One more, and to make things more fun, we'll mix joins, aggregates functions, and grouping, too. Ready? Back in Chapter 13, I challenged you to find all order numbers with a value of 1000 or more. Those results are useful, but what would be even more useful is the names of the customers who placed orders of at least that amount. So, write a SQL statement that uses joins to return the customer name (`cust_name`) from the `Customers` table and the total price of every order from the `OrderItems` table. Here's a hint: To join those tables, you'll also need to include the `Orders` table (because `Customers` is not related directly to `OrderItems`, `Customers` is related to `Orders`, and `Orders` is related to `OrderItems`). Don't forget `GROUP BY` and `HAVING` and be sure to sort the results by customer name. You can use simple equijoin or ANSI `INNER JOIN` syntax for this one. Or, if you are feeling brave, try writing it both ways.

This page intentionally left blank